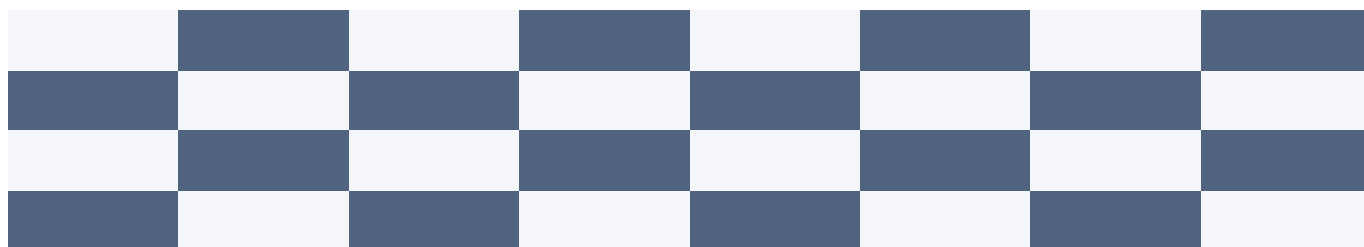


JOGO DE XADREZ

Tutorial Completo do Codigo



Motor completo de regras, Inteligencia Artificial e quatro cameras

Sumario

- 1. Arquitetura geral do projeto**
- 2. index.html: estrutura da interface**
- 3. estilo.css: layout e responsividade**
4. motor.js: modelo de estado do xadrez
5. Criacao das pecas e movimentos
6. Geracao e filtragem de movimentos legais
7. Peoes, en passant e promocao
8. Cavalos, pecas deslizantes e rei
9. Roque e direitos de roque
10. Deteccao de casas atacadas e xeque
11. Aplicacao de movimentos e historico
12. Resultados e regras de empate
13. Repeticao, 50 e 75 lances
14. ia.js: arquitetura da Inteligencia Artificial
15. Nivel Facil
16. Minimax e poda Alpha-Beta
17. Funcao de avaliacao da IA
18. Ordenacao de movimentos e anti-ciclo
19. renderizador3d.js: Canvas e cameras
20. Projecao do tabuleiro e clique nas casas
21. Desenho das pecas 3D
22. app.js: integracao da aplicacao
23. Selecao e execucao de jogadas
24. Fluxo da IA e controle de tempo
25. Promocao e eventos especiais
26. Finalizacao, empate e desistencia
27. Sons e controles do navegador
28. Persistencia, ranking e estatisticas
29. Mapa das principais funcoes
30. Fluxo completo de uma jogada
31. Decisoese de projeto e extensao

1. Arquitetura geral do projeto

O projeto foi dividido por responsabilidade. Essa separacao reduz o acoplamento entre as regras, a Inteligencia Artificial, a interface e a renderizacao. O HTML define os elementos visuais; o CSS cuida do layout; motor.js representa as regras; ia.js escolhe jogadas; renderizador3d.js desenha o tabuleiro e as pecas; app.js conecta tudo e controla o fluxo da partida.

A aplicacao nao depende de bibliotecas externas. Toda a logica de xadrez, busca da IA e renderizacao em Canvas foi implementada no proprio projeto.

Arquivo	Responsabilidade principal	Linhas
index.html	Estrutura da interface, menus e paineis	1-1019
css/estilo.css	Layout, temas, responsividade e cameras	1-4364
js/motor.js	Regras e estado completo do xadrez	1-5232
js/ia.js	IA, Minimax, poda e avaliacao	1-2170
js/renderizador3d.js	Canvas, projecao, cameras e pecas 3D	1-8027
js/app.js	Interface, sons, fluxo, ranking e eventos	1-7188

2. index.html: estrutura da interface

O arquivo index.html funciona como a estrutura semantica da aplicacao. Ele possui a tela de configuracao, seletores de modo, cor, dificuldade e camera, botoes de inicio e telas auxiliares. Na area de partida ficam o seletor de camera, o tabuleiro/Canvas, paineis de jogador, historico, capturas e acoes como reivindicar empate, oferecer empate, desistir e voltar ao menu.

Os elementos recebem identificadores que app.js consulta com document.getElementById. Dessa maneira, o JavaScript nao precisa criar toda a interface do zero; ele atualiza elementos previamente definidos no HTML.

3. estilo.css: layout e responsividade

O CSS concentra a identidade visual, dimensoes do tabuleiro, paineis, botoes, modais, estados de selecao e regras responsivas. Tambem existem estilos relacionados aos quatro enquadramentos, ao Canvas e aos elementos que precisam permanecer legiveis em diferentes tamanhos de navegador.

A responsividade e importante porque o renderizador calcula seu tamanho a partir da area disponivel. O CSS define o espaco; o JavaScript mede esse espaco e adapta o desenho.

4. motor.js: modelo de estado do xadrez

A classe XadrezMotor, em motor.js, é o núcleo das regras. O construtor começa na linha 290 e prepara propriedades como tabuleiro, turno, direitos de roque, possibilidade de en passant, histórico e estruturas de repetição. O método reiniciar(), a partir da linha 408, restaura a posição inicial.

O método clonar(), a partir da linha 658, cria uma cópia independente do estado. Essa operação é fundamental para a IA, que precisa simular movimentos sem alterar a partida real.

Elemento	Linhas aproximadas	Função
constructor()	290-407	Inicializa o estado.
reiniciar()	408-657	Monta a posição inicial e zera controles.
clonar()	658-874	Copia o estado para simulações.
pecaEm()	897-924	Le uma peça em determinada casa.

5. Criação das peças e movimentos

A função criarPeca(), nas linhas 142-169 de motor.js, padroniza o objeto que representa cada peça. criarMovimento(), nas linhas 170-259, monta uma estrutura de movimento contendo origem, destino, peça e opções especiais.

Essa representação única permite que movimentos comuns, capturas, roques, en passant e promoções passem pelo mesmo fluxo de aplicação. Em vez de espalhar várias formas de dados, o motor trabalha com um formato consistente.

6. Geração e filtragem de movimentos legais

movimentosLegais(), a partir da linha 925, produz todos os movimentos permitidos para uma cor. movimentosDaCasa(), na linha 1016, restringe a consulta a uma origem específica, o que é usado pela interface quando o jogador seleciona uma peça.

O motor primeiro gera movimentos pseudo-legais e depois elimina os que deixam o próprio rei em xeque. Essa separação simplifica a implementação: cada peça cuida da geometria do seu movimento, enquanto a validação final cuida da segurança do rei.

Método	Linhas	Papel
movimentosLegais()	925-1015	Lista todos os movimentos legais da cor.
movimentosDaCasa()	1016-1045	Lista destinos legais de uma peça selecionada.
explicarMovimentoInvalido()	1046-1467	Produz explicação em português para

		tentativa invalida.
_movimentosPseudoLegais()	1468-1690	Gera candidatos antes do filtro de xeque.

7. Peoes, en passant e promocao

_movimentosPeao(), linhas 1691-2077, concentra as particularidades da peca mais complexa em regras especiais: avanco simples, avanco duplo inicial, captura diagonal, en passant e geracao de opcoes de promocao.

O estado guarda a casa relevante para en passant apenas pelo periodo em que a captura e permitida. Na promocao, o movimento carrega a informacao da peca escolhida, permitindo que a aplicacao substitua o peao no destino.

8. Cavalos, pecas deslizantes e rei

_movimentosCavalo(), linhas 2078-2210, usa os oito deslocamentos em L e nao precisa verificar casas intermediarias. _movimentosDeslizantes(), linhas 2211-2377, e reutilizado para torre, bispo e dama percorrendo uma direcao ate encontrar a borda ou uma peca.

_movimentosRei(), linhas 2378-2539, trata os passos de uma casa e agrega os candidatos de roque. O filtro legal posterior garante que o rei nao termine atacado.

9. Roque e direitos de roque

_movimentosRoque(), linhas 2540-2813, verifica direitos registrados, presenca da torre, casas vazias e seguranca das casas percorridas pelo rei. _atualizarDireitosRoque(), linhas 4055-4301, remove os direitos quando rei ou torre se movem ou quando uma torre relevante e capturada.

Separar geracao e manutencao dos direitos impede que um rei retorne a casa inicial e volte a poder rocar indevidamente.

10. Deteccao de casas atacadas e xeque

casaAtacada(), linhas 2814-3355, responde se uma casa e controlada pela cor adversaria. encontrarRei(), linhas 3356-3440, localiza o rei da cor desejada. estaEmXeque(), linhas 3441-3488, combina as duas operacoes.

Essas funcoes sao usadas em diversas regras: filtro de movimentos legais, roque, deteccao de xeque e resultado final.

11. Aplicacao de movimentos e historico

aplicarMovimento(), a partir da linha 3489, e a entrada publica para executar uma jogada valida.

_aplicarSemValidar(), a partir da linha 3568, efetua as alteracoes concretas no tabuleiro e e util em simulacoes internas.

O movimento atualiza captura, promocao, en passant, deslocamento da torre no roque, contador de meio-lances, turno, historico e chave de repeticao. Centralizar essas atualizacoes evita divergencia entre jogadas do usuario e da IA.

12. Resultados e regras de empate

resultado(), linhas 4302-4411, verifica se a partida deve terminar. A avaliacao considera ausencia de movimentos legais, xeque-mate, afogamento, material insuficiente e empates automaticos por repeticao ou contagem de lances.

podeReivindicarEmpateRepeticao(), podeReivindicarEmpate50Lances() e motivosEmpateReivindicavel() separam as condicoes que dependem de uma acao do jogador.

Metodo	Linhas
resultado()	4302-4411
podeReivindicarEmpateRepeticao()	4412-4440
podeReivindicarEmpate50Lances()	4441-4461
motivosEmpateReivindicavel()	4462-4513
materialInsuficiente()	4514-4682

13. Repeticao, 50 e 75 lances

_chavePosicao(), linhas 4683-4803, cria uma representacao da posicao usada na contagem de repeticoes. O calculo inclui informacoes que alteram a legalidade futura, como turno, direitos de roque e en passant legalmente disponivel.

_registrarPosicao(), linhas 5021-5231, incrementa a ocorrencia da configuracao atual. O contador de meio-lances e atualizado na aplicacao dos movimentos e serve para as regras de 50 e 75 lances.

14. ia.js: arquitetura da Inteligencia Artificial

A classe XadrezIA esta em ia.js. O construtor inicia na linha 65 e recebe o nivel. escolherMovimento(), a partir da linha 98, e a entrada principal. Ela consulta os movimentos legais, normaliza o nome do nivel e direciona o processamento.

No Intermediario a profundidade principal e 2; no Avancado, 3. A busca utiliza copias do motor, aplica movimentos hipoteticos e avalia a posicao resultante sem modificar a partida visivel.

15. Nivel Facil

_escolherFacil(), linhas 386-576, implementa o comportamento do nivel mais acessivel. Em vez de explorar a arvore com a mesma profundidade dos niveis superiores, ele escolhe usando criterios mais leves, permitindo maior variacao e erros estrategicos.

A existencia de um caminho separado para o nível Fácil evita gastar processamento desnecessário e cria uma diferença real de desafio.

16. Minimax e poda Alpha-Beta

`_minimax()`, linhas 577-1026, implementa a busca adversarial. Em um nível, a IA tenta maximizar sua avaliação; no seguinte, assume que o adversário escolherá a resposta que minimiza esse valor. O processo continua até a profundidade definida ou até uma posição terminal.

Os parâmetros alfa e beta armazenam limites conhecidos. Quando um ramo não pode mais melhorar o resultado já encontrado, a poda Alpha-Beta interrompe sua exploração. Isso reduz a quantidade de posições avaliadas sem alterar a decisão Minimax para a mesma ordem e profundidade.

17. Função de avaliação da IA

`avaliar()`, linhas 1508-1723, converte uma posição em um valor numérico do ponto de vista da cor da IA. A avaliação combina material e bônus posicionais. `_bonusPosicional()`, linhas 1724-1950, valoriza a colocação das peças em casas mais adequadas ao seu papel.

Uma função de avaliação é necessária porque a busca normalmente termina antes do fim real da partida. O número estimado representa quão favorável é a posição naquele ponto da árvore.

18. Ordenação de movimentos e anti-ciclo

`_ordenarMovimentos()`, linhas 1951-2170, coloca candidatos promissores em ordem que favorece a poda. A classe ainda possui `deveReivindicarEmpate()`, ajustes anti-ciclo e verificações para evitar desfazer repetidamente o próprio movimento ou mover a mesma peça sem ganho.

Esses mecanismos deixam o comportamento mais natural e reduzem loops que poderiam surgir em avaliadores de profundidade limitada.

19. `renderizador3d.js`: Canvas e câmeras

`renderizador3d.js` implementa a classe responsável pelo Canvas. O construtor inicia na linha 35. `definirCamera()`, linha 264, troca o enquadramento; `renderizar()`, linha 304, recebe o estado visual; `_ajustarCanvas()`, linha 378, sincroniza resolução e área disponível.

`_configuracao()`, a partir da linha 565, fornece parâmetros específicos para cada câmera. A `Camera Superior` usa uma projeção mais direta, enquanto as câmeras de perspectiva transformam as coordenadas das casas para produzir profundidade aparente.

20. Projeção do tabuleiro e clique nas casas

`_projetar()`, linhas 802-929, transforma coordenadas lógicas do tabuleiro em coordenadas de tela. As funções `_viewRow()`, `_viewCol()`, `_actualRow()` e `_actualCol()` convertem a orientação de acordo com o lado da câmera.

Para detectar cliques, o renderizador guarda a geometria das casas projetadas. `_pontoNoPoligono()` testa se o cursor está dentro de uma casa; `_casaNoPonto()`, a partir da linha 7757, retorna a casa correspondente e `_clicar()`, linha 7875, encaminha a seleção para `app.js`.

21. Desenho das pecas 3D

`_desenharPecas()`, linha 3322, percorre as pecas visiveis e chama `_desenharPeca()`, linha 3744. O desenho nao usa imagens prontas: cada modelo e construido com Canvas, gradientes e perfis geometricos.

Ha rotinas especializadas para peao, torre, bispo, dama, rei e cavalo. Funcoes auxiliares como `_gradienteMaterial()`, `_basePeca()`, `_corpoPerfil()`, `_anel()` e `_esfera()` reutilizam componentes comuns para manter consistencia visual.

Peca / componente	Inicio aproximado
<code>_peao()</code>	5323
<code>_torre()</code>	5418
<code>_bispo()</code>	5796
<code>_dama()</code>	6082
<code>_rei()</code>	6413
<code>_cavalo()</code>	6665
<code>_gradienteMaterial()</code>	4233
<code>_basePeca()</code>	4344

22. app.js: integracao da aplicacao

`app.js` e a camada de orquestracao. Ele referencia `XadrezMotor`, `XadrezIA` e `RenderizadorXadrez3D`, le os controles da pagina, administra sons, estados de tela, modais e armazenamento local.

`criarTabuleiroDOM()`, linha 2227, prepara a representacao interativa usada pela aplicacao. `renderizarTabuleiro()`, linha 2502, sincroniza selecao, destaques e renderizador 3D. `renderizarTudo()`, linha 3293, atualiza tabuleiro e paineis em conjunto.

23. Selecao e execucao de jogadas

`clicarCasa()`, linhas 3350-3688, recebe a casa clicada. Ele verifica se existe partida, de quem e o turno, se a casa possui uma peca selecionavel e quais destinos sao legais. Se houver mais de uma opcao para o mesmo destino por causa de promocao, o fluxo chama `abrirPromocao()`.

`executarMovimento()`, linhas 3916-4181, aplica o movimento no motor, aciona efeitos, atualiza a interface, verifica o resultado e agenda a proxima jogada quando ela pertence a IA.

24. Fluxo da IA e controle de tempo

`agendarIA()`, linhas 4303-4397, programa a proxima acao automatica respeitando o estado atual da partida. `iniciarCalculoIA()`, linhas 4398-4605, chama a IA para escolher um movimento e garante que o retorno ainda pertence a mesma partida antes de aplicar o lance.

Esse cuidado e importante porque o usuario pode voltar ao menu, pausar ou iniciar outra partida enquanto existe uma acao programada. Identificar a partida evita que um calculo antigo seja aplicado ao estado novo.

25. Promocao e eventos especiais

`abrirPromocao()`, linhas 3689-3915, apresenta as opcoes de promocao quando um peao alcança a ultima fileira. Depois que uma opcao e escolhida, o movimento correspondente segue para `executarMovimento()`.

`mostrarAvisoCaptura()`, linhas 4182-4302, controla avisos temporarios de captura. Sons especificos sao acionados de acordo com o tipo do movimento, incluindo captura, roque, promocao e xeque.

26. Finalizacao, empate e desistencia

`iniciarPartida()`, linhas 4606-4924, coleta configuracoes, cria o motor, instancia IAs conforme o modo, prepara a camera, limpa estados anteriores e inicia o fluxo. `traduzirResultado()`, linha 4925, converte codigos internos em textos; `finalizarResultado()`, linha 5143, registra e apresenta o resultado.

`reivindicarEmpate()`, linha 5476, consulta os motivos permitidos pelo motor. `oferecerEmpate()`, linha 5627, implementa a negociacao com a IA. `desistir()`, linha 5844, confirma a acao e finaliza com vitoria adversaria.

27. Sons e controles do navegador

O mapa de sons e criado em `app.js` e associa nomes logicos a arquivos WAV. `tocarSom()`, linha 1031, reproduz o efeito apenas quando o som esta habilitado. `alternarSom()`, linha 6576, muda o estado e e usado pelo botao e pela tecla F8.

`cameraAplicar()`, linha 1690, envia a camera escolhida ao renderizador e atualiza os controles. Os eventos de teclado tambem mapeiam 1 a 4 para cameras e F8 para audio.

28. Persistencia, ranking e estatisticas

A persistencia e encapsulada em uma classe de armazenamento dentro de `app.js`. `_ler()`, linha 433, recupera JSON do `localStorage`; `_salvar()`, linha 531, grava os dados; `registrarPartida()`, linha 570, atualiza o perfil e adiciona o resultado ao historico.

`estatisticas()`, linha 805, retorna os dados de um jogador; `ranking()`, linha 846, ordena os perfis; `historico()`, linha 890, fornece as partidas registradas. O sistema soma 3 pontos por vitoria, 1 por empate e 0 por derrota.

29. Mapa das principais funcoes

A tabela abaixo funciona como um indice tecnico para localizar rapidamente os principais pontos do projeto. Os intervalos correspondem ao arquivo fornecido.

Arquivo	Funcao / metodo	Linhas	Responsabilidade
motor.js	XadrezMotor.constructor()	290-407	Cria o estado do jogo.
motor.js	reiniciar()	408-657	Restaura a posicao inicial.
motor.js	clonar()	658-874	Copia o estado para simulacao.
motor.js	movimentosLegais()	925-1015	Gera movimentos legais.
motor.js	movimentosDaCasa()	1016-1045	Destinos de uma peca.
motor.js	explicarMovimentoInvalido())	1046-1467	Explica tentativa invalida.
motor.js	_movimentosPeao()	1691-2077	Peao, en passant e promocao.
motor.js	_movimentosRoque()	2540-2813	Valida roques.
motor.js	casaAtacada()	2814-3355	Detecta controle de casa.
motor.js	estaEmXeque()	3441-3488	Detecta xeque.
motor.js	aplicarMovimento()	3489-3567	Entrada para executar lance.
motor.js	resultado()	4302-4411	Determina resultado da partida.
motor.js	materialInsuficiente()	4514-4682	Detecta empate por material.
ia.js	escolherMovimento()	98-385	Entrada principal da IA.
ia.js	_escolherFacil()	386-576	Logica do nivel Facil.
ia.js	_minimax()	577-1026	Busca Minimax com poda.
ia.js	deveReivindicarEmpate()	1027-1138	Decide reivindicacao.
ia.js	avaliar()	1508-1723	Avalia uma posicao.
ia.js	_bonusPosicional()	1724-1950	Bonus de posicionamento.
ia.js	_ordenarMovimentos()	1951-2170	Ordena candidatos.
renderizador3d.js	definirCamera()	264-303	Troca camera.
renderizador3d.js	renderizar()	304-342	Recebe estado visual.
renderizador3d.js	_projetar()	802-929	Projeta coordenadas no

			Canvas.
renderizador3d.js	_desenharTabuleiro()	2103-3099	Desenha casas e destaques.
renderizador3d.js	_desenharPecas()	3322-3653	Desenha conjunto de pecas.
renderizador3d.js	_desenharPeca()	3744-4232	Despacha o modelo da peca.
renderizador3d.js	_casaNoPonto()	7757-7874	Converte clique em casa.
app.js	tocarSom()	1031-1278	Reproduz efeitos sonoros.
app.js	cameraAplicar()	1690-1887	Aplica camera.
app.js	renderizarTabuleiro()	2502-2860	Atualiza tabuleiro.
app.js	clicarCasa()	3350-3688	Processa selecao humana.
app.js	abrirPromocao()	3689-3915	Escolha de promocao.
app.js	executarMovimento()	3916-4181	Executa e trata um lance.
app.js	agendarIA()	4303-4397	Agenda turno automatico.
app.js	iniciarCalculoIA()	4398-4605	Calcula lance da IA.
app.js	iniciarPartida()	4606-4924	Configura e inicia partida.
app.js	finalizarResultado()	5143-5475	Finaliza e registra resultado.
app.js	reivindicarEmpate()	5476-5626	Processa reivindicacao.
app.js	oferecerEmpate()	5627-5843	Oferta de empate contra IA.
app.js	desistir()	5844-5936	Processa desistencia.
app.js	ranking()	6275-6374	Exibe ranking.
app.js	alternarSom()	6576-6674	Liga/desliga audio.

30. Fluxo completo de uma jogada

1. O usuario clica em uma casa. renderizador3d.js converte o ponto da tela na casa logica e chama o callback configurado pela aplicacao.
2. clicarCasa() consulta movimentosDaCasa() no motor. O motor gera candidatos e remove os que deixariam o rei em xeque.
3. Ao escolher um destino, executarMovimento() chama aplicarMovimento(). O motor altera o estado, registra historico, atualiza roque, en passant, contadores e repeticao.

4. `app.js` toca os sons apropriados e chama `renderizarTudo()`. O renderizador recebe o novo estado e redesenha tabuleiro, peças e destaques.
5. `resultado()` verifica xeque-mate, afogamento e empates. Se a partida continuar e o próximo turno for da IA, `agendarIA()` inicia a próxima decisão.
6. A IA clona o estado, explora movimentos hipotéticos e retorna um movimento. O mesmo `executarMovimento()` é usado, garantindo que usuário e IA passem pelo mesmo motor de regras.

31. Decisões de projeto e extensão

A principal decisão arquitetural é manter o motor independente da interface. Isso permite testar regras e simular posições sem depender do Canvas. A IA recebe o mesmo tipo de estado usado na partida real, e o renderizador apenas representa os dados que recebe.

Outra decisão importante é desenhar as peças em Canvas em vez de depender de modelos ou imagens externas. Isso torna o projeto autocontido e permite quatro câmeras com o mesmo conjunto de peças.

Para expandir o projeto, novos recursos podem ser adicionados preservando essa separação. Exemplos: relógio de xadrez pode ser tratado em `app.js`; novos critérios de avaliação entram em `ia.js`; importação/exportação de posições pertence ao motor e a camada de interface; novos estilos de câmera ficam no renderizador.

- Mantenha regras oficiais dentro de `motor.js`.
- Mantenha critérios de decisão automática dentro de `ia.js`.
- Mantenha desenho e projeção dentro de `renderizador3d.js`.
- Mantenha DOM, áudio, persistência e fluxo de telas dentro de `app.js`.
- Ao adicionar um recurso, reutilize `aplicarMovimento()` para não criar caminhos de jogada inconsistentes.